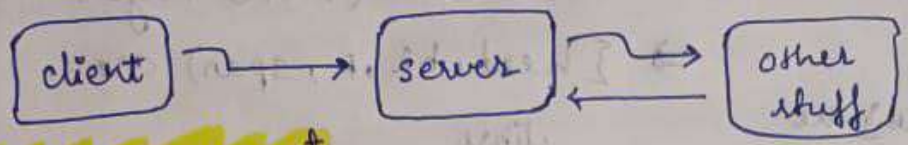
2nd April : Chai aur React (Part 1)

– Jaani

1



1. sends a request
2. get a response

Client has to collect some data and send it to server and get some data from server and shows it to the user.

Client can be web, mobile etc.

→ But we focus only on web here, and browsers only understand HTML, CSS, JS.

⇒ AJAX, JQuery helps us build website in such a way, that the entire website doesn't reload, everytime any changes is made. This was done by updating dom references. But at scale, where there are too many references and too many people 5M +, the website felt laggy.

→ Facebook had ghost notifications, bcs at scale (too many references) browser were not allowing to remove the notifications, the browser had most control.

→ Now we took control of DOM (now named Virtual DOM)

→ Virtual DOM (a DOM that lives in memory)

DOM (Document Object Model)

⇒ Difference b/w DOM and Virtual DOM

DOM	Virtual DOM (Fibre algo)
⇒ {head, h1, p, span} are called nodes	⇒ {head, h1, p, span} are fibre
⇒ update	⇒ reconciliation [React uses diffing algo, just like github]

⇒ Frontend — client | Server

Client side — HTML + CSS + JS

Server side — React ⇒ manages UI, handles state, updates DOM efficiently

Here everything is happening on server side, after HTML CSS, JS is loaded.
React starts running, and creates virtual DOM, and renders UI on the screen.

⇒ Now we use vite for creating a React app.

Step 1: run "npm create vite@latest"

Step 2: Give a project name

Step 3: Select a framework: React

Step 4: Select JS for now.

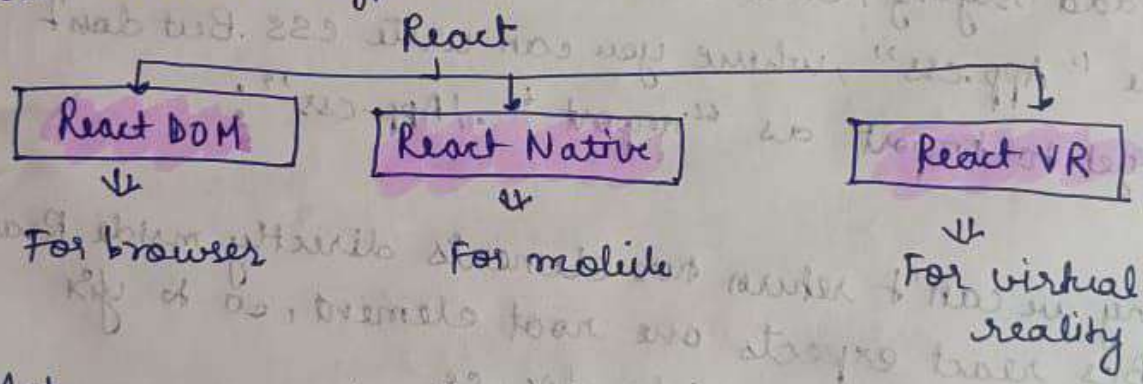
Step 5 : Say "yes" to install with npm.

Folder structure

project
- ↳ node_modules
- ↳ public folder (has all assets etc)
- ↳ src (where my App lives)
  - ↳ main.jsx (main JS file)
    - ↳ App.jsx (main component)
- ↳ gitignore
- ↳ index.html (entry point)
- ↳ package.json (dependencies)
- ↳ Readme.MD
- ↳ vite.config.js (configuration for vite)

~~Inside "App.jsx"~~

Also React has types or renderers ⇒



⇒ Make components in different files and export them, and then to use it, import in "main.jsx" file.

⇒ Inside "main.jsx",

```
import {createRoot} from "react-dom/client";
import App from "./App.jsx";
createRoot(document.getElementById("root")).render(
<App/>
);
```

Note : Vite is a dev dependency. Vite expects a 'index.html' in root and " main.jsx " in src.

Assignment : Research about Jsx, render, Vite

Inside "App.jsx"

```
function App(){
  return (
    <div>
    <h1> Jaani is writing the code </h1>
  </div>
  );
}
export default App;
```

To add styling, either add in "App.jsx" or make a new file " App.css", where you can write CSS. But don't forget to import as "import " ./App.css";

Why we can't return two elements directly inside React.
⇒ Bcs react expects one root element, so to fix the error, wrap inside div. eg

```
<div>
  <h1> Hello </h1>
  <p> World </p>
</div>
```

* Use fragments <> and </>, a cleaner way

Array ⇒

```
const shows = [
    { id : 1 ,
      title : "Virtual DOM",
      time : "04:00 PM",
      hall : "Hall C",
    }
function App() {
  return (
    <div>
      <h1> Hello </h1>
      <h2> I am show { show . title } at time [show . time]
                                                  </h2>
    </div>
```

or to show the data,

```
  return (
    <div>
      <h1> Hello </h1>
      <section class Name = "grid" >
      { shows . map ( (show) ⇒ {
        <article>
          <p className = "tag" > { show . hall } </p>
      <h3> { show . title } </h3>
      <p class Name = "muted" > { show . time } </p>
    </article>
  )) }
  </section>
</div>
```

Note : To write JS use curly braces { } .

Assignment: To write code to fetch data from API and store it in a variable. Use "Freeapi" for API's.

6

Props (properties) are objects. Props are how we pass data from one component to another.

eg:

```
function Shell ({ title, children }) {
return (
  <section>
    <p> Reusable Shell </p>
    <h2> { title } </h2>
    { children }
  </section>
)
}

function ~~Ava~~ AvatarCard ({ avatar, level = "Rookie" }) {
return (
  <article>
    <div> { avatar.initials } </ /div>
    <~~div~~ h3> { avatar.name } </ ~~/div~~ /h3>
    <p> Level : { level } </ /p>
  </article>
);
}
fun App () {
return (
  <>
<section>
  { avatars.map( ( avatar ) => (
    <AvatarCard avatar = { avatar } />
  ) }
</ /section> </ />
```

props

to display data

Also you can put Avatar Card as a component and import it wherever needed.

Also you can use a component like:

```
<Shell title="Batman">
  <div>
    <h1> . . . . </h1>
  </div>
</Shell>
```

Children helps us create wrapper components.
It is the data that you put between a component's tags.

const [count, setCount] = useState(0);

→ default value

When you pass fun' in default value, it called "lazy initialization"
This line means create a variable called "count", with starting value 0, and give a fun^n "setCount" to update it.

const [count, setCount] = useState(0);

→ default value

data holder

data changer (setter fun)

To make your own,

const [value, setValue] = useState(5);

To read value,

<h1> value </h1>

To update, const increase = () ⇒ { setValue(value + 1);

not recommended.

To make for decrement,

```
const decrease = () => {
    setValue(value - 1); // not a preferred method.
};
```

Hook : A function that lets us use React's feature like state, effect etc) inside functional components.

Common hooks :

useState → manage state
useEffect → handle side effects
useRef, use context etc

UseEffect is a hook used for running code after render

UseState,

```
const [posts, setPosts] = useState([]);
const [status, setStatus] = useState("Idle");
const [seconds, setSeconds] = useState(10);
```

// Here posts, status and seconds are piece of code.

```
const addData = () => {
    setPosts([
        ...posts,
        "Jaani"
        "Chaicode"
    ]);
};
```

1. Takes current posts
2. Spreads old values
3. Adds new items
4. Update state → trigger re-render

```
return (
    <div>
        <h1> use Effect </h1>
    </div>
```

UseEffect has a callback, dependency array and cleanup

```
useEffect(() ⇒
    return () ⇒ {
        // cleanup
    }
},[])
```

(Here return is optional)

This is runs at "on component mount", for the first time and second time depends on dependency array [] if any change in [], it would reload.

Creating a timer using useEffect ⇒

```
useEffect(() ⇒ {
    const timerId = setInterval(() ⇒ {
        setSeconds((current) ⇒ Math.max(current - 1, 0);
    },1000);
    return () ⇒ {
        clearInterval(timerId);
    };
},[]);
```

callback runs every 1000ms (1 sec)

component mounts (effects runs only once)

How it happens,

1. useEffect runs once
2. setInterval starts
3. Every 1 sec → setState
4. Re-render
5. Do cleanup when component unmounts, to avoid memory leaks.

How to do API fetching and state handling, 10

In the below code, we fetch posts when the component loads and also manage (loading/success) state.

"Code"

```
useEffect(() => {
    const controller = new AbortController()
    async fun loadPost() {
      try {
        setStatus("Loading")
      const response = await fetch("https://json ...", {signal: controller.signal})
      const data = await response.json()
      setPosts(data)
      setStatus("Success")
      } catch (error){
        // TODO
      }
      loadPost();
    }, [])
```

search about this

The error part here,

```
~~} catch{error}~~
catch(error) {
  if (error.name === "AbortError") {
      console.log("Fetch aborted");
  } else {
      setStatus("error");
  }
}
```

Do cleanup, return () => { controller.abort();

You should abort if, user navigates away, bcs abort stops the request when the component is gone so it doesn't update something that does not exists now.

To abort,
return () => controller.abort();

Assignment: Use FreeApI.app, Can we fetch data, and make projects. Atleast 3-4

Clean up fun is called on un-mount (on leaving page in our example)

We have completed topics:

1. Component Architecture
2. Reusable component
3. children
4. props
5. default value in props
6. data display
7. key
8. State, useEffect, useState
9. Conditional rendering

Follow: jaaaari404

Happy learning!